

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using `fork()`. 3. Execute the command in the child process using an appropriate `exec()` system call. 4. Allow the parent process to wait for the child using `wait()`. 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (`uname`, `lscpu`, `lsblk`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char command[100];

    printf("Enter Linux command: ");
    scanf("%s", command);

    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        printf("Child PID: %d\n", getpid());
        execlp(command, command, NULL);

        printf("Command execution failed\n");
    }
    else {
        // Parent process
        printf("Parent PID: %d\n", getpid());
        wait(NULL);
        printf("Child process completed\n");
    }

    return 0;
}
```

```
nishanth@LAPTOP-CU1RFQM  X  +  v
nishanth@LAPTOP-CU1RFQML:~$ vi program.c
nishanth@LAPTOP-CU1RFQML:~$ gcc program.c -o program
nishanth@LAPTOP-CU1RFQML:~$ ./program
Enter Linux command: ls
Parent PID: 605
Child PID: 608
hello  hello.c  program  program.c
Child process completed
```